

Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации

*Ордена Трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования*

**«Московский технический университет связи и информатики»
(МТУСИ)**

Лабораторная работа № 3

по дисциплине “Информационные технологии и программирование”

Выполнила:

студентка группы БВТ2301

Чиняева Алиса Юрьевна

Проверил:

Харрасов К.Р.

Лабораторная работа № 3

Хэш-таблица – это структура данных, которая используется для хранения пар «ключ-значение». Она основана на идее хэш-функции, которая преобразует ключ в индекс массива, где хранится значение.

Хэш-функция должна быть быстрой и однозначной, то есть каждому ключу должен соответствовать уникальный индекс. Если два разных ключа преобразуются в один и тот же индекс, то это называется коллизией. Коллизии могут быть разрешены различными способами, например, с помощью метода цепочек или открытой адресации.

В Java хэш-таблицы реализованы в виде классов `HashMap` и `Hashtable`. Они имеют похожий интерфейс, но различаются в том, что `HashMap` не является потокобезопасным, а `Hashtable` является.

Задание 1:

1. Создайте класс `HashTable`, который будет реализовывать хэштаблицу с помощью метода цепочек.
2. Реализуйте методы `put(key, value)`, `get(key)` и `remove(key)`, которые добавляют, получают и удаляют пары «ключ-значение» соответственно.
3. Добавьте методы `size()` и `isEmpty()`, которые возвращают количество элементов в таблице и проверяют, пуста ли она.

Для начала нам необходим класс под каждый элемент нашей цепочки по индексу в хэш-таблице. Создадим класс `Element<K, V>`, где `K` и `V` - дженерики(обобщения). В нем мы храним ключ и значение `K key`, `V value`, а также объявляем методы для получения ключа и значения `getKey()`, `getValue()` и метод для смены значения `setValue()`. Ключ является константой, поэтому поменять мы его не можем.

```
private static class Element<K, V> {
    private final K key;
    private V value;

    public Element(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public V getValue() {
        return value;
    }

    public void setValue(V value) {
        this.value = value;
    }
}
```

Далее мы объявляем массив цепочек и переменную для размера и добавляем конструктор.

```
private int capacity = 10;
private LinkedList<Element<K, V>>[] table;
private int size;

@SuppressWarnings("unchecked")
public HashTable() {
    table = new LinkedList[capacity];
    size = 0;
}
```

Используется аннотация `@SuppressWarnings("unchecked")`, так как в Java не разрешено напрямую создавать обобщённые массивы.

Метод `hash(K key)` вычисляет индекс для ключа, используя его хэш-код и деление по модулю на размер массива.

```
private int hash(K key) {
    return Math.abs(key.hashCode()) % table.length;
}
```

Добавляем методы для работы с хеш-таблицей, такие как `put(K key, V value)`, `get(K key)`, `remove(K key)`. Также добавляем методы для получения кол-ва элементов в хеш-таблице и проверки на пустоту.

```

public void put(K key, V value) {
    int index = hash(key);

    if (table[index] == null) {
        table[index] = new LinkedList<>();
    }

    for (Element<K, V> element : table[index]) {
        if (element.getKey().equals(key)) {
            element.setValue(value);
            return;
        }
    }

    table[index].add(new Element<>(key, value));
    size++;
}

```

```

public V get(K key) {
    int index = hash(key);

    if (table[index] != null) {
        for (Element<K, V> element : table[index]) {
            if (element.getKey().equals(key)) {
                return element.getValue();
            }
        }
    }

    return null;
}

```

```

public void remove(K key) {
    int index = hash(key);

    if (table[index] != null) {
        for (Element<K, V> element : table[index]) {
            if (element.getKey().equals(key)) {
                table[index].remove(element);
                size--;
                return;
            }
        }
    }
}

public int size() {
    return size;
}

public boolean isEmpty() {
    return size == 0;
}

```

Использование описанного класса в Main.

```
public class Main {
    public static void main(String[] args) {
        HashTable<String, Integer> myHashTable = new HashTable<>();
        myHashTable.put(key:"me", value:22);
        myHashTable.put(key:"sister", value:17);
        myHashTable.put(key:"mother", value:40);
        myHashTable.put(key:"father", value:45);
        myHashTable.put(key:"brother", value:3);

        System.out.println("Моей сестре " + myHashTable.get(key:"sister") + " лет");

        System.out.println("Элементов в таблице с братом: " + myHashTable.size());
        // i don't have one :)
        myHashTable.remove(key:"brother");
        System.out.println("Элементов в таблице без брата: " + myHashTable.size());

        System.out.println("Пустота таблицы: " + myHashTable.isEmpty());
        myHashTable.remove(key:"me");
        myHashTable.remove(key:"mother");
        myHashTable.remove(key:"father");
        myHashTable.remove(key:"sister");
        System.out.println("Пустота таблицы после чистки: " + myHashTable.isEmpty());

        // get them back
        myHashTable.put(key:"me", value:22);
        myHashTable.put(key:"sister", value:17);
        myHashTable.put(key:"mother", value:40);
        myHashTable.put(key:"father", value:45);
        myHashTable.put(key:"brother", value:3);
    }
}
```

```
Моей сестре 17 лет
Элементов в таблице с братом: 5
Элементов в таблице без брата: 4
Пустота таблицы: false
Пустота таблицы после чистки: true
```

Задание 2 Работа с встроенным классом HashMap.

Вариант 10: Реализация хэш-таблицы для хранения данных о заказах в *ресторане*. Ключом будет *номер столика* или *заказа*, а значением - *объект класса Order*, содержащий информацию о блюдах, стоимости и времени заказа. Необходимо реализовать операции вставки, поиска и удаления заказа по номеру столика.

Для реализации нашей задачи создадим объект класса Order. В нем будут храниться блюда, чек и время заказа, также опишем конструктор с параметрами и геттеры.

```
class Order {  
    private String dishes;  
    private int cost;  
    private String orderTime;  
  
    public Order(String dishes, int cost, String orderTime) {  
        this.dishes = dishes;  
        this.cost = cost;  
        this.orderTime = orderTime;  
    }  
  
    public String getDishes() {  
        return dishes;  
    }  
  
    public int getCost() {  
        return cost;  
    }  
  
    public String getOrderTime() {  
        return orderTime;  
    }  
}
```

Дальше опишем основной класс Restaurant с параметрами ключ-значение, где значение - объект класса Order, а также методы для добавления, удаления и вывода значения (объекта класса Order).

```

public Restaurant() {
    this.orders = new HashMap<>();
}

public void addOrder(int tableNumber, Order order) {
    orders.put(tableNumber, order);
}

public Order getOrder(int tableNumber) {
    if (orders.containsKey(tableNumber)) {
        return orders.get(tableNumber);
    } else {
        System.out.println("Столик " + tableNumber + " нет заказов.");
        return null;
    }
}

public void removeOrder(int tableNumber) {
    if (orders.containsKey(tableNumber)) {
        orders.remove(tableNumber);
    } else {
        System.out.println("❌ столика " + tableNumber + " нет заказов.");
    }
}

public void printAllOrders() {
    if (orders.isEmpty()) {
        System.out.println("Нет активных заказов.");
    } else {
        for (Integer tableNumber : orders.keySet()) {
            Order order = orders.get(tableNumber);
            System.out.println("Столик " + tableNumber + " - блюда: "
                + order.getDishes() + ", чек: "
                + order.getCost() + ", время заказа: "
                + order.getOrderTime());
        }
    }
}

```

Применяю все вышеописанные методы и классы в main.

```

Run [Debug]
public static void main(String[] args) {
    Restaurant restaurant = new Restaurant();

    restaurant.addOrder(tableNumber:1, new Order(dishes:"Кола, Пицца Липперони", cost:550, orderTime:"12:30"));
    restaurant.addOrder(tableNumber:2, new Order(dishes:"Бургер, Салат, Трюфель, Чай", cost:1500, orderTime:"16:00"));
    restaurant.addOrder(tableNumber:3, new Order(dishes:"Суши, Соевый соус, Чай матча", cost:3200, orderTime:"20:00"));

    Order order2 = restaurant.getOrder(tableNumber:2);
    System.out.println("Столик 2 - блюда: "
        + order2.getDishes() + ", чек: "
        + order2.getCost() + ", время заказа: "
        + order2.getOrderTime());

    restaurant.removeOrder(tableNumber:2);

    restaurant.printAllOrders();
}

```

```
• (base) aliror@DESKTOP-HNFQP65:~/projects1/unik/itip/lab3$ java Restaurant
Столик 2 - блюда: Бургер, Салат, Трюфель, Чай, чек: 1500, время заказа: 16:00
Столик 1 - блюда: Кола, Пицца Пипперони, чек: 550, время заказа: 12:30
Столик 3 - блюда: Суши, Соевый соус, Чай матча, чек: 3200, время заказа: 20:00
```